🚀 Lecture 02.09: Logical Operators & Short-Circuit Evaluation

📌 Executive Architectural Summary

Logical operators operate across truth values according to Boolean algebra. In Python, `and` and `or` do not merely return booleans; they return the **actual operand object** that resolved the expression via short-circuit evaluation.

🔬 Core Engineering Mechanics

1. Truth Tables

| A | B | $A \wedge B$ (`and`) | $A \vee B$ (`or`) | $eg A$ (`not`) | | :---: | :---: | :---: | :---: | :---: | | `True` | `True` | `True` | `True` | `False` | | `True` | `False` | `False` | `True` | `False` | | `False` | `True` | `False` | `True` | `True` | | `False` | `False` | `False` | `False` | `True` |

2. Short-Circuit Evaluation Guarantees

- `A and B` : If A is false, A is immediately returned without evaluating B. If A is true, B is evaluated and returned.
- `A or B` : If A is true, A is immediately returned without evaluating B. If A is false, B is evaluated and returned.

💻 Production Python Implementation

```
# 1. Standard Boolean Logic
a, b = True, False
print(f"a and b: {a and b}")
print(f"a or b:  {a or b}")
print(f"not a:   {not a}")

# 2. Short-Circuit Guarded Evaluation Pattern
class NetworkService:
    def is_connected(self):
        print("--> Expensive network ping evaluated!")
        return True
```

```
service = None

# If service is None, service.is_connected() is NEVER called (No AttributeError!)
safe_check = (service is not None) and service.is_connected()
print("Safe Guarded Check Result:", safe_check)
```